

Smart Logistic

Documentación de Diseño — Arquitectura, Flujos, Modelo de Datos y Despliegue

Generado a partir del código fuente y los ADRs del repositorio
24 de septiembre de 2026

Documentación de Diseño — Smart Logistic

Este documento consolida, con diagramas UML (notación Mermaid, renderizable directamente en GitHub), la arquitectura, los flujos de ejecución, el modelo de datos y la guía de despliegue del sistema **tal como está implementado hoy** en el repositorio. Complementa — no reemplaza — los ADRs individuales en `docs/architecture/00` a `06`, que documentan el *porqué* de cada decisión; aquí se documenta el *qué* y el *cómo*, con foco visual.

1. Requerimientos tomados y cómo se resuelven

1.1 Requerimientos de negocio (funcionales)

#	Requerimiento	Cómo se resuelve
R1	Gestionar clientes (personas naturales y empresas) con RUT único, datos de contacto, condición de pago y prioridad comercial	Agregado <code>Cliente</code> en <code>customer-service</code> , con invariantes de negocio encapsuladas (ver §3.1) y API REST <code>/api/v1/clientes</code> (ADR bounded context Clientes, §3 de <code>03- ... md</code>)
R2	Cada cliente puede tener múltiples direcciones de entrega, con coordenadas geográficas para planificación de rutas futura	Entidad interna <code>Direccion</code> dentro del agregado <code>Cliente</code> , con invariante de “exactamente una dirección principal”
R3	Las direcciones deben validarse contra la división político-administrativa oficial de Chile (región/provincia/comuna)	<code>geo-catalog-service</code> : catálogo de 16 regiones / 56 provincias / 346 comunas con códigos oficiales INE, consumido por <code>customer-service</code> vía HTTP (ADR 05)
R4	Determinar si una fecha es día hábil chileno (fines de semana + feriados legales) para uso futuro en agendamiento de entregas	<code>CalendarioChileno</code> en <code>shared-kernel</code> : cálculo algorítmico (fijos, Pascua vía Meeus/Jones/Butcher, traslados Ley 19.668), expuesto también como API en <code>geo-catalog-service</code> (<code>/api/v1/dias-habiles/**</code> , <code>/api/v1/feriados</code>)
R5	El RUT nunca debe ser clave técnica interna (regla dura del negocio)	Cada agregado usa <code>UUID</code> (<code>ClienteId</code>) como PK; <code>Rut</code> es un <code>Value Object</code> con dígito verificador, único a nivel de negocio (<code>UNIQUE</code> en BD) pero nunca FK entre servicios
R6	Un cliente no se elimina físicamente; se conserva su historial comercial	<code>Cliente.desactivar()</code> cambia <code>estado</code> a <code>INACTIVO</code> (soft-delete lógico); no existe operación de borrado físico en el dominio
R7	El sistema debe escalar y desplegarse en apps móviles a futuro, y en una nube aún no definida	API REST versionada (<code>/api/v1/ ...</code>) y stateless en cada servicio; empaquetado 100% en contenedores Docker sin dependencias a servicios propietarios de un proveedor cloud específico

#	Requerimiento	Cómo se resuelve
R8	Interfaz web para operar el sistema	<code>frontend/</code> (Angular 22, standalone + signals + Angular Material): listado/alta/detalle de clientes, gestión de direcciones, explorador del catálogo geográfico

1.2 Requerimientos no funcionales

#	Requerimiento	Cómo se resuelve
NF1	Escalado independiente por dominio (Clientes cambia poco, otros dominios futuros cambian mucho)	Estilo microservicios desde el día 1, uno por <i>bounded context</i> , cada uno con su propio ciclo de build/deploy (ADR-001)
NF2	Aislamiento de fallos: una caída de un servicio de referencia no debe tumbar la escritura de datos core	<i>Circuit breaker</i> + <i>retry</i> + caché Caffeine en la integración <code>customer-service</code> → <code>geo-catalog-service</code> (ADR 05); un 503 explícito (<code>ComunaCatalogNoDisponibleException</code>) en vez de fallo silencioso
NF3	Consistencia entre servicios sin transacciones distribuidas	<i>Database per Service</i> + Transactional Outbox + RabbitMQ (ADR-002); eventos versionados (<code>schemaVersion</code>) e idempotencia vía <code>eventId</code>
NF4	Portabilidad (Docker, cualquier nube)	Cada servicio tiene su <code>Dockerfile</code> multi-stage; <code>infra/docker-compose.yml</code> orquesta el stack completo localmente sin dependencia de un orquestador propietario
NF5	Trazabilidad de procedencia de datos de referencia	Catálogo geográfico documenta explícitamente su fuente (<code>knxroot/bdcut-cl</code>) y limita el alcance no cubierto (Ley 20.983) — ver §4 de <code>04- ... md</code>
NF6	Pipeline de pruebas rápido en el día a día, sin forzar Docker en cada <code>mvn test</code>	Pirámide de 4 niveles (unitarias → Testcontainers → smoke Docker Compose → BDD Cucumber), detallada en ADR 06
NF7	Auditoría de cambios (quién/cuándo)	<code>AuditInfo</code> (VO en <code>shared-kernel</code>) embebido en cada agregado; header <code>X-Usuario</code> / query param <code>usuario</code> en las mutaciones REST (placeholder hasta implementar Identidad y Acceso)

1.3 Explícitamente fuera de alcance (declarado, no un olvido)

- Autenticación/autorización real (Spring Security + JWT es un punto de extensión documentado, no implementado).
- Persistencia o versionado histórico del catálogo geográfico (se trata como snapshot vigente).
- Regla de feriado adicional Ley 20.983 (Fiestas Patrias condicional).
- Bounded contexts de Inventario, Pedidos, Recurrencias, Reprogramación, Planificación Logística y Pagos (roadmap iterativo, ver `00-overview.md` §5).

2. Vista de arquitectura (C4 — Contenedores)

```
flowchart TB
    subgraph Cliente_Externo["Usuario"]
        Browser["Navegador Web"]
    end

    subgraph Docker["Stack Docker (infra/docker-compose.yml)"]
        FE["frontend<br/>Angular 22 + Nginx<br/>:4200 → :80"]
        CS["customer-service<br/>Spring Boot 3.3 / Java 21<br/>:9081"]
        GS["geo-catalog-service<br/>Spring Boot 3.3 / Java 21<br/>:9082"]
        PG1[("postgres<br/>customer_service<br/>:5432")]
        PG2[("postgres-geo<br/>geo_catalog_service<br/>:5433")]
        MQ[{"rabbitmq<br/>:5672 / mgmt :15672"}]
    end

    Browser -- "HTTP :4200" --> FE
    FE -- "proxy /api/v1/clientes/**" --> CS
    FE -- "proxy /api/v1/** (resto)" --> GS
    CS -- "JDBC" --> PG1
    CS -- "HTTP síncrono<br/>(Retry + CircuitBreaker + Cache)" --> GS
    CS -- "AMQP (outbox → exchange<br/>clientes.events)" --> MQ
    GS -- "JDBC" --> PG2

    classDef svc fill:#e8f0fe,stroke:#4285f4;
    classDef db fill:#fce8e6,stroke:#ea4335;
    class FE,CS,GS svc;
    class PG1,PG2,MQ db;
```

Notas de la vista:

- `frontend` nunca llama directo a `customer-service / geo-catalog-service` en producción: Nginx actúa como *reverse proxy* (`nginx.conf`), evitando CORS y no exponiendo los backends al navegador (ver §6.2).
- `RabbitMQ` hoy no tiene consumidores propios en el repositorio (no existen aún Pedidos/Inventario); `customer-service` publica a través del *outbox* dejando la integración lista para el próximo bounded context consumidor.
- `geo-catalog-service` no publica eventos (es de solo lectura, ADR bounded context §9).

3. Diagrama de clases — Dominio

3.1 customer-service (bounded context Clientes)

```
classDiagram
class Cliente {
    -ClienteId id
    -Rut rut
    -TipoCliente tipoCliente
    -String nombre
    -List~Telefono~ telefonos
    -List~Correo~ correos
    -CondicionPago condicionPago
    -PrioridadComercial prioridadComercial
    -EstadoCliente estado
    -List~Direccion~ direcciones
    -AuditInfo auditInfo
    +crear(rut, tipo, nombre, ...)$ Cliente
    +reconstruir(...)$ Cliente
    +actualizarDatosDeContacto(...)
    +agregarDireccion(...) DireccionId
    +actualizarDireccion(...)
    +marcarDireccionComoPrincipal(...)
    +desactivar(usuario, ahora)
    +reactivar(usuario, ahora)
    +eventosPendientes() List~DomainEvent~
}

class Direccion {
    -DireccionId id
    -String calle
    -String numero
    -Comuna comuna
    -Coordenadas coordenadas
    -String referencia
    -boolean esPrincipal
    +actualizarDatos(... )~package~
    +marcarComoPrincipal()~package~
    +desmarcarComoPrincipal()~package~
}

class ClienteId { +UUID value +nuevo()$ +de(String)$ }
class DireccionId { +UUID value +nuevo()$ +de(String)$ }
class Rut { +long numero +String dv +formatoCanónico() +of(String)$ }
class Comuna { <<record>> +String código +String nombre +Region region }
class Region { <<enum>> ARICA_Y_PARINACOTA ... MAGALLANES +porCodigoIne(int)$ }
class Coordenadas { <<record>> +double latitud +double longitud }
class Telefono { <<record>> +String numero }
class Correo { <<record>> +String direccion }
class CondicionPago { <<enum>> CONTADO CREDITO_7_DIAS CREDITO_15_DIAS CREDITO_30_DIAS }
class PrioridadComercial { <<enum>> ESTANDAR PREFERENTE VIP }
class EstadoCliente { <<enum>> ACTIVO INACTIVO }
class TipoCliente { <<enum>> PERSONA_NATURAL EMPRESA }
```

```

class AuditInfo { +String createdBy +Instant createdAt +String updatedBy +Instant updatedAt
class DomainEvent { <<abstract>> +UUID eventId +Instant occurredOn +String aggregated

Cliente "1" *-- "1" ClienteId
Cliente "1" *-- "1" Rut
Cliente "1" *-- "0..*" Telefono
Cliente "1" *-- "0..*" Correo
Cliente "1" *-- "1" AuditInfo
Cliente "1" *-- "0..*" Direccion : agregado interno
Direccion "1" *-- "1" DireccionId
Direccion "1" *-- "1" Comuna
Direccion "1" *-- "1" Coordenadas
Comuna "1" → "1" Region
Cliente ..> DomainEvent : publica

class ClienteCreadoEvent
class ClienteActualizadoEvent
class DireccionAgregadaEvent
class ClienteDesactivadoEvent
DomainEvent <|-- ClienteCreadoEvent
DomainEvent <|-- ClienteActualizadoEvent
DomainEvent <|-- DireccionAgregadaEvent
DomainEvent <|-- ClienteDesactivadoEvent

```

Invariantes clave (impuestas por el propio agregado, no por la capa de persistencia):

1. **Cliente** requiere al menos un teléfono o correo para poder ser **ACTIVO** (**ClienteSinContactoException**).
2. Existe **exactamente una** **Direccion** marcada **esPrincipal** cuando hay ≥ 1 dirección; la primera dirección agregada se marca principal automáticamente.
3. **Direccion** solo es mutable a través de métodos *package-private*, invocados exclusivamente desde **Cliente** (encapsulamiento del agregado).
4. Toda mutación relevante encola un **DomainEvent** recuperado por la capa de aplicación (**eventosPendientes()**), nunca publicado directamente por el dominio (el dominio no conoce RabbitMQ).

3.2 geo-catalog-service (bounded context Catálogo Geográfico)

```

classDiagram
class Region { <<record>> +int codigo +String nombre }
class Provincia { <<record>> +int codigo +String nombre +int regionCodigo }
class Comuna { <<record>> +int codigo +String nombre +int provinciaCodigo +int regionCodigo }
class Feriado { <<record, shared-kernel>> +LocalDate fecha +String nombre +TipoFeriado }
class TipoFeriado { <<enum>> FIJO MOVIL_PASCUA MOVIL_LEY_19668 AD_HOC }
class CalendarioChileno { <<domain service, shared-kernel>> +feriadosDelAnio(int) +esFeriado(LocalDate) }

Region "1" ← "0..*" Provincia : region_codigo
Provincia "1" ← "0..*" Comuna : provincia_codigo
CalendarioChileno ..> Feriado : calcula (sin persistencia)
Feriado → TipoFeriado

```

Notas: a diferencia de Clientes, aquí las claves primarias **son** el código oficial INE (excepción documentada y justificada en `04-bounded-context-catalogo-geografico.md` §3 , porque el código INE es emitido por una autoridad externa estable, no por este sistema). `CalendarioChileno` no tiene estado ni tabla propia: es una función pura invocada en cada request.

4. Modelo de datos (diagrama entidad-relación)

4.1 Esquema `customer-service` (PostgreSQL, Flyway `V1__init_schema.sql`)

```
erDiagram
    CLIENTE ||--o{ CLIENTE_TELEFONO : tiene
    CLIENTE ||--o{ CLIENTE_CORREO : tiene
    CLIENTE ||--o{ DIRECCION : tiene
    CLIENTE {
        UUID id PK
        BIGINT rut_numero
        VARCHAR rut_dv
        VARCHAR tipo_cliente
        VARCHAR nombre
        VARCHAR condicion_pago
        VARCHAR prioridad_comercial
        VARCHAR estado
        TIMESTAMPTZ created_at
        VARCHAR created_by
        TIMESTAMPTZ updated_at
        VARCHAR updated_by
    }
    CLIENTE_TELEFONO {
        UUID cliente_id FK
        VARCHAR numero PK
    }
    CLIENTE_CORREO {
        UUID cliente_id FK
        VARCHAR direccion_correo PK
    }
    DIRECCION {
        UUID id PK
        UUID cliente_id FK
        VARCHAR calle
        VARCHAR numero
        VARCHAR comuna_codigo
        VARCHAR comuna_nombre
        VARCHAR region
        DOUBLE latitud
        DOUBLE longitud
        VARCHAR referencia
        BOOLEAN es_principal
    }
    OUTBOX_EVENT {
        UUID id PK
        VARCHAR aggregate_id
        VARCHAR event_type
        INT schema_version
        TEXT payload
        TIMESTAMPTZ occurred_on
        TIMESTAMPTZ published_at "NULL = pendiente"
    }
```

- `uk_cliente_rut` `UNIQUE(rut_numero, rut_dv)` : garantiza unicidad de negocio del RUT sin usarlo como PK.
- `idx_outbox_event_pendientes` (índice parcial `WHERE published_at IS NULL`): optimiza el *polling* del `OutboxDispatcher`.
- `OUTBOX_EVENT` no tiene FK hacia `CLIENTE` : es una tabla de infraestructura de mensajería, desacoplada intencionalmente del modelo de dominio.

4.2 Esquema `geo-catalog-service` (PostgreSQL, Flyway `v1 – v4`)

```
erDiagram
    REGION ||--o{ PROVINCIA : contiene
    PROVINCIA ||--o{ COMUNA : contiene
    REGION {
        INTEGER codigo PK "código INE, 1-16"
        VARCHAR nombre
    }
    PROVINCIA {
        INTEGER codigo PK "código INE, 3 dígitos"
        VARCHAR nombre
        INTEGER region_codigo FK
    }
    COMUNA {
        INTEGER codigo PK "código INE, 4-5 dígitos"
        VARCHAR nombre
        INTEGER provincia_codigo FK
    }
```

- Datos sembrados vía `V2__seed_regiones.sql` / `V3__seed_provincias.sql` / `V4__seed_comunas.sql` (346 comunas), generados desde `docs/data/bdcut-cl-raw.csv` (fuente pública `knxroot/bdcut-cl`).
- Sin tabla de feriados: se calculan en memoria (ver §3.2).
- **Database per Service**: no existe ninguna FK entre el esquema de `customer-service` y el de `geo-catalog-service`; la única integración es HTTP (§5.3).

5. Diagramas de secuencia (flujos principales)

5.1 Crear cliente (con publicación de evento vía Transactional Outbox)

```
sequenceDiagram
    actor U as Usuario
    participant FE as frontend (Angular)
    participant CTRL as ClienteController
    participant APP as ClienteApplicationService
    participant DOM as Cliente (agregado)
    participant REPO as ClienteRepositoryAdapter
    participant DB as PostgreSQL (customer_service)
    participant OUT as OutboxDispatcher (scheduler)
    participant MQ as RabbitMQ

    U->>FE: Completa formulario "Nuevo cliente"
    FE->>CTRL: POST /api/v1/clientes
    CTRL->>APP: ejecutar(ComandoCrearCliente)
    APP->>APP: valida RUT único (existePorRut)
    APP->>DOM: Cliente.crear(rut, ...)
    DOM->>DOM: valida invariante (≥1 contacto)
    DOM->>APP: Cliente + evento ClienteCreadoEvent (en memoria)
    APP->>REPO: guardar(cliente)
    REPO->>DB: INSERT cliente + INSERT outbox_event (misma transacción)
    APP->>APP: eventPublisherPort.publicar(eventos)
    Note over APP,DB: publicar() solo persiste en outbox_event;<br/>no hay llamada de rec
    APP->>CTRL: ClienteId
    CTRL->>FE: 201 Created {id}
    FE->>U: Confirmación + navega a detalle

    loop cada 2s (fixedDelay configurable)
        OUT->>DB: SELECT pendientes (published_at IS NULL)
        OUT->>MQ: convertAndSend(clientes.events, "cliente.creado.v1", payload)
        OUT->>DB: UPDATE outbox_event SET published_at = now()
    end
end
```

Por qué esta forma: separar el *commit* transaccional (paso 8) del envío a RabbitMQ (bucle inferior) evita el problema clásico de “dual write” (guardar en BD y publicar en el broker no son atómicos si se hacen directamente). Si el proceso cae entre ambos pasos, el evento queda `published_at = NULL` y se reintenta en el siguiente ciclo — entrega *at-least-once*, por lo que los futuros consumidores deben deduplicar por `eventId`.

5.2 Agregar dirección (integración resiliente con Catálogo Geográfico)

```
sequenceDiagram
    actor U as Usuario
    participant FE as frontend
    participant CTRL as ClienteController
    participant APP as ClienteApplicationService
    participant PORT as ComunaCatalogPort
    participant ADAPT as GeoCatalogHttpComunaCatalogAdapter
    participant CACHE as Caffeine Cache (comunas)
    participant GEO as geo-catalog-service
    participant DOM as Cliente (agregado)

    U->>FE: Autocompleta comuna + coordenadas
    FE->>CTRL: POST /clientes/{id}/direcciones
    CTRL->>APP: ejecutar(ComandoAgregarDireccion)
    APP->>PORT: buscarPorCodigo(codigoComuna)
    PORT->>ADAPT: (implementación HTTP, perfil ≠ local)

    alt En caché
        ADAPT->>CACHE: hit
        CACHE->>ADAPT: Comuna
    else Cache miss
        ADAPT->>GEO: GET /api/v1/comunas/{codigo}
        alt 200 OK
            GEO->>ADAPT: ComunaCatalogoResponse
            ADAPT->>CACHE: almacenar
        else 404
            GEO->>ADAPT: 404
            ADAPT->>APP: Optional.empty()
            APP->>CTRL: 400 ComunaDesconocidaException
        else timeout / 5xx / circuito abierto
            Note over ADAPT: Retry (3 intentos, 200ms)<br/>luego CircuitBreaker fallback
            ADAPT->>APP: ComunaCatalogNoDisponibleException
            APP->>CTRL: 503 Service Unavailable
        end
    end

    ADAPT->>APP: Optional[Comuna]
    APP->>DOM: cliente.agregarDireccion(..., comuna, ...)
    DOM->>DOM: aplica invariante dirección principal
    APP->>CTRL: DireccionId
    CTRL->>FE: 201 Created
```

Distinción crítica de esta integración (ADR 05): un 404 (comuna inexistente) es una respuesta *válida* del dominio → **400 Bad Request**. Una falla de infraestructura (timeout, 5xx, circuito abierto) es un error operacional distinto → **503 Service Unavailable**. El sistema nunca confunde “no existe” con “no disponible”.

5.3 Consulta de catálogo geográfico y verificación de día hábil (desde el frontend)

```
sequenceDiagram
    actor U as Usuario
    participant FE as CatalogoGeograficoPage (Angular)
    participant NGX as Nginx (reverse proxy)
    participant GEOC as GeoCatalogController
    participant SVC as GeoCatalogApplicationService
    participant REPO as ComunaRepositoryAdapter
    participant DB as PostgreSQL (geo_catalog_service)
    participant CAL as CalendarioChileno (shared-kernel)

    U->>FE: Selecciona región → provincia → comuna
    FE->>NGX: GET /api/v1/regiones/{codigo}/comunas
    NGX->>GEOC: proxy_pass (mismo request)
    GEOC->>SVC: listarComunasDeRegion(codigo)
    SVC->>REPO: buscar en BD
    REPO->>DB: SELECT ... WHERE region_codigo
    DB->>REPO: filas
    REPO->>SVC: List~Comuna~
    SVC->>GEOC: List~Comuna~
    GEOC->>FE: 200 OK [ComunaResponse]

    U->>FE: Ingresa fecha a verificar
    FE->>NGX: GET /api/v1/dias-habiles/verificar?fecha= ...
    NGX->>GEOC: proxy_pass
    GEOC->>SVC: esDiaHabil(fecha)
    SVC->>CAL: esDiaHabil(fecha)
    CAL->>SVC: boolean (calculado, sin BD)
    SVC->>GEOC: boolean
    GEOC->>FE: 200 OK {fecha, esDiaHabil}
```

6. Vista de despliegue

6.1 Diagrama de despliegue (contenedores, puertos, redes)

```
flowchart LR
    subgraph Host["Máquina host / nodo"]
        subgraph net["Red Docker: infra_default"]
            direction TB
            FE["frontend:80<br/>nginx:1.27-alpine"] ::: web
            CS["customer-service:9081<br/>eclipse-temurin:21-jre"] ::: app
            GS["geo-catalog-service:9082<br/>eclipse-temurin:21-jre"] ::: app
            PG["postgres:5432<br/>postgres:16-alpine"] ::: data
            PGG["postgres-geo:5432<br/>postgres:16-alpine"] ::: data
            RMQ["rabbitmq:5672,15672<br/>rabbitmq:3.13-mgmt"] ::: data
        end
    end

    Host_Port_4200(["host:4200"]) → FE
    Host_Port_9081(["host:9081"]) → CS
    Host_Port_9082(["host:9082"]) → GS
    Host_Port_5432(["host:5432"]) → PG
    Host_Port_5433(["host:5433"]) → PGG
    Host_Port_15672(["host:15672"]) → RMQ

    FE -.depends_on.-> CS
    FE -.depends_on.-> GS
    CS -.depends_on: service_healthy.-> PG
    CS -.depends_on: service_healthy.-> RMQ
    CS -.depends_on: service_healthy.-> GS
    GS -.depends_on: service_healthy.-> PGG

    classDef web fill:#e8f0fe,stroke:#4285f4;
    classDef app fill:#fef7e0,stroke:#f9ab00;
    classDef data fill:#fce8e6,stroke:#ea4335;
```

6.2 Cómo desplegar

Opción A — Stack completo con Docker Compose (recomendado, replica producción)

Requisitos: Docker Desktop (o Docker Engine + Compose plugin) en ejecución.

```
cd C:\git\smart-logistic\infra
docker compose up -d --build
```

Esto:

1. Construye las imágenes de `customer-service` y `geo-catalog-service` (multi-stage: Maven+JDK 21 para compilar, JRE 21 liviano para ejecutar, usuario no-root `appuser`).
2. Construye la imagen del `frontend` (multi-stage: Node 22 para compilar Angular en modo `production`, Nginx 1.27 para servir estáticos + *reverse proxy*).

3. Levanta `postgres`, `postgres-geo` y `rabbitmq`, esperando sus *healthchecks* antes de arrancar los servicios de aplicación (`depends_on: condition: service_healthy`).
4. Ejecuta migraciones Flyway automáticamente al iniciar cada servicio Spring Boot.

Verificación post-despliegue:

```
docker ps                                # todos los contenedores "Up"/"healthy"
curl http://localhost:9081/actuator/health # customer-service → {"status":"UP"}
curl http://localhost:9082/actuator/health # geo-catalog-service → {"status":"UP"}
curl http://localhost:4200                 # frontend → 200 OK
```

Puertos configurables (variables de entorno, valores por defecto entre paréntesis):

`CUSTOMER_SERVICE_PORT` (9081), `GEO_CATALOG_SERVICE_PORT` (9082), `FRONTEND_PORT` (4200),
`CUSTOMER_POSTGRES_PORT` (5432), `GEO_POSTGRES_PORT` (5433), `RABBITMQ_PORT` (5672),
`RABBITMQ_MANAGEMENT_PORT` (15672).

Importante — un solo consumidor del puerto: si se usa `ng serve` (modo desarrollo) en paralelo al contenedor `frontend`, ambos compiten por el puerto 4200 y el navegador puede terminar hablando con el proceso equivocado (el dev server llama directo a `localhost:9081/9082`, sin proxy ni CORS). Usar **uno u otro**, no ambos a la vez.

Opción B — Desarrollo local ligero (sin Docker para los servicios de aplicación)

Perfil Spring `local`: usa H2 en memoria, deshabilita Flyway y el *outbox*/health-check de RabbitMQ.

```
cd services\customer-service
mvn -DskipTests package
java -jar target\customer-service.jar --spring.profiles.active=local

cd services\geo-catalog-service
mvn -DskipTests package
java -jar target\geo-catalog-service.jar --spring.profiles.active=local

cd frontend
npm install
npx ng serve    # http://localhost:4200, apunta a localhost:9081/9082 (environment.ts)
```

Opción C — Desarrollo con infraestructura real (Postgres + RabbitMQ vía Docker, apps en JVM local)

```
docker compose -f infra\docker-compose.yml up -d postgres postgres-geo rabbitmq
cd services\customer-service; mvn -DskipTests package; java -jar target\customer-service.jar
cd services\geo-catalog-service; mvn -DskipTests package; java -jar target\geo-catalog-service.jar
```

Despliegue en la nube (orientación general)

El sistema no está acoplado a un proveedor específico (requisito NF4/R7): cada servicio es una imagen Docker autocontenida que expone `/actuator/health` para *liveness/readiness probes*. Para producción se recomienda:

- Orquestador de contenedores (Kubernetes gestionado, Azure Container Apps, ECS, etc.) con un *Deployment* por servicio y *readiness probe* sobre `/actuator/health`.
- PostgreSQL gestionado (una instancia por *bounded context*, preservando *Database per Service*) en lugar de los contenedores `postgres` / `postgres-geo` de desarrollo.
- RabbitMQ gestionado o cluster propio con *dead-letter exchange* configurado (ADR-002).
- Variables de entorno equivalentes a las de `docker-compose.yml` (`SPRING_DATASOURCE_URL`, `GEOCATALOGSERVICE_BASEURL`, credenciales RabbitMQ) inyectadas vía *secrets* del orquestador, no *hardcoded*.
- El *reverse proxy* de `frontend` (Nginx) puede sustituirse por un *API Gateway/Ingress* cuando exista más de un canal cliente (ADR-001, "consecuencias").

6.3 Estrategia de pruebas antes de desplegar

Ver detalle completo en `06-estrategia-de-pruebas.md`. Resumen ejecutable:

```
mvn test                    # Nivel 1: unitarias, sin Docker (se ejecutan en el IDE)
mvn verify                  # Nivel 2: + integración con Testcontainers
.\infra\docker-compose-smoke-test.ps1 # Nivel 3: smoke E2E sobre imágenes
mvn -f qa-automation/pom.xml verify `
  "-Dcustomer.service.base-url=http://localhost:18081" `
  "-Dgeo.catalog.service.base-url=http://localhost:18082" # Nivel 4: BDD/Cucumber
```

7. Trazabilidad de este documento

Fuente	Verificado contra
Modelo de dominio (§3)	Código fuente en <code>services/*/src/main/java/ ... /domain/model</code>
Modelo de datos (§4)	Migraciones Flyway reales (<code>V1__init_schema.sql</code> de ambos servicios)
Flujos (§5)	<code>ClienteApplicationService</code> , <code>OutboxDispatcher</code> , <code>GeoCatalogHttpComunaCatalogAdapter</code> , <code>GeoCatalogController</code>
Despliegue (§6)	<code>infra/docker-compose.yml</code> , <code>Dockerfile</code> de cada servicio y del frontend, <code>frontend/nginx.conf</code>
Requerimientos (§1)	<code>docs/architecture/00-overview.md</code> y ADRs 01, 02, 05; código de invariantes en <code>Cliente.java</code>

Este documento debe actualizarse en cada nueva iteración del roadmap (`00-overview.md` §5) que agregue un *bounded context* nuevo (Inventario, Pedidos, etc.).